Hardware / Link Layer Specification

PN532 Binding of the Transport Layer Link Layer Interface (LLI)

Version 0.1 – Draft

July 24, 2026

Abstract

This document binds the abstract Link Layer Interface (LLI) defined in *smartlock_transport_layer_spec_v2.pdf* (Section 2) to a concrete controller: the NXP PN532 NFC front-end operating as an ISO/IEC 14443-4 Type A PICC in 106 kbps Passive Card Emulation Mode. It specifies the exact PN532 host commands, byte-level frame formats, and firmware-confirmed behaviors used to implement each LLI primitive. All PN532-specific claims below have been verified against NXP UM0701-02 (PN532 User Manual) and AN133910 (PN532 Application Note); none of this content should be assumed to generalize to a different controller. Should the Lock's NFC front-end ever change, only this document and its corresponding driver need to be replaced.

Contents

1 Scope

This document is the sole owner of PN532-specific detail in the system. It implements, and only implements, the five LLI primitives required by the transport layer: `LLI_Activate`, `LLI_ReceiveAPDU`, `LLI_SendAPDU`, `LLI_GetLinkStatus`, `LLI_Abort`. It does not define or alter APDU framing, the instruction set, protocol state machine, timing policy (T_{HS}), or status word semantics – those remain normative in the transport layer document.

1.1 Target Configuration

Parameter	Value
Role	Target (PICC), Card Emulation Mode
Modulation	Type A, 106 kbps, Passive
Protocol	ISO/IEC 14443-4 (ISO-DEP)
Host Interface	SPI / I2C / HSU (deployment-specific; not further constrained here)

2 LLI Primitive Bindings

2.1 `LLI_Activate()` → `TgInitAsTarget (0x8C)`

The ESP32 host issues `TgInitAsTarget`, configuring the PN532 for Type A Passive Card Emulation (SENS_RES, NFCID1, SEL_RES, and ATS historical bytes are supplied as command parameters). The PN532 firmware then autonomously handles, without further host involvement:

- `SENS_REQ` response
- `SDD_REQ` anti-collision cascade
- `SEL_REQ` completion
- `RATS` reception and automatic `ATS` transmission using the configured historical bytes

This is confirmed firmware behavior per NXP UM0701-02 §4: *"ATS will be sent automatically to the reader which has sent a RATS."* `LLI_Activate()` returns success once `TgInitAsTarget` completes and the ATS has been sent (i.e. the PN532 has reached its internal ISO-DEP-activated target state).

2.2 `LLI_ReceiveAPDU(timeout)` → `TgGetData (0x86)`

Input: [`0xD4`, `0x86`]   Output: [`0xD5`, `0x87`, Status, C-APDU (up to 262 bytes)]

2.2.1 Status Byte Mapping

Confirmed against NXP AN133910 §3.2.2. The driver MUST translate these PN532-native codes into the transport layer's abstract error taxonomy as shown:

PN532 Code	PN532 Meaning	LLI Abstract Result
`0x00`	Success	Valid C-APDU returned
`0x01`	Timeout – target has not answered	`TIMEOUT`
`0x02`	CRC error (contactless UART)	`FRAME_INTEGRITY_ERROR`
`0x03`	Parity error (contactless UART)	`FRAME_INTEGRITY_ERROR`
other	Any other non-zero status	`FRAME_INTEGRITY_ERROR` (fail closed)

Cryptographic safeguard: on any status other than `0x00`, the driver MUST return an error to the transport layer *without* passing partial or corrupted data upward. As specified in the transport layer document, a `FRAME_INTEGRITY_ERROR` during *Handshake Pending* or *Secure Session* triggers the secure-erase routine; a corrupted frame must never reach signature verification or AES-GCM decryption.

2.3 `LLI_SendAPDU(bytes)` → `TgSetData (0x8E)`

$$\text{Input: } [\texttt{0xD4}, \texttt{0x8E}, \text{DataOut}[\ldots]] \qquad \text{Output: } [\texttt{0xD5}, \texttt{0x8F}, \text{Status}]$$

A non-`0x00` status on the `TgSetData` response indicates the R-APDU was not successfully queued for transmission; the driver MUST surface this as a send failure to the caller rather than assuming delivery.

2.3.1 Waiting Time Extension

Confirmed per NXP UM0701-02 §4: while acting as an ISO/IEC 14443-4 Type A PICC, the PN532 autonomously issues an `S(WTX)` request to the Initiator if the host controller does not supply the R-APDU within the negotiated Frame Waiting Time (FWT). This satisfies the transport layer's requirement (Section 8.2 of the transport document) that WTX be handled transparently within `LLI_SendAPDU` – **no additional host logic is required** to trigger WTX on this controller.

Observed bounds on this behavior, relevant to capacity planning:

- Default maximum single WTX-extended window on PN532 deployments has been observed on the order of ∼1 second (community-reported, configurable via RF configuration parameters governing `ATR_RES`/timeout behavior).
- Initiator-side stacks commonly cap the number of accepted WTX requests per exchange; the PN532 side does not control this limit.

Given Ed25519 signing on the ESP32 completes in low single-digit milliseconds, WTX is not expected to trigger during the handshake phase with this controller. If a future revision moves signing to the ATECC608A secure element, the added I^2C transaction latency should be re-measured against this budget.

2.4 `LLI_GetLinkStatus()` → `TgGetTargetStatus (0x8A)`

The driver polls `TgGetTargetStatus` and maps its response to the LLI's three-value status:

PN532 Status	Meaning	LLI Mapping
Target activated	Link is up and selected	`ACTIVE`
`0x80`	Target released by initiator	`RELEASED`
Communication error	Bus-level failure reading status	`ERROR`

Recommended polling interval: every 100–200 ms while in *Handshake Pending* or *Secure Session*, and additionally, synchronously, immediately before any state-changing action (e.g. before actuating the physical lock mechanism). This interval is a PN532-specific recommendation, chosen to keep detection latency for RF tear-away well below the T_{HS} handshake timeout defined by the transport layer, while avoiding excessive bus traffic. A different controller's Hardware/Link Layer document may recommend a different interval based on its own status-query cost and latency characteristics.

2.5 `LLI_Abort()` → `InRelease` / Re-issue `TgInitAsTarget`

Abort is implemented by issuing an `InRelease` (or equivalent deselect) sequence and returning the PN532 to a state where `TgInitAsTarget` can be re-issued to restart from *IDLE*. This is invoked by the transport layer's secure-erase routine on `CMD_SESSION_ABORT`, handshake timeout, or any detected integrity failure, so that the RF front-end and the cryptographic session state are torn down together.

3 Frame Size Conformance

The PN532's Short APDU support is confirmed per NXP UM0701-02 to satisfy the bounds assumed by the transport layer without reduction:

Direction	PN532-Confirmed Limit
External PCD → PN532 (as PICC)	Up to 261 bytes (CLA/INS/P1/P2/Lc + 255 data bytes + Le)
PN532 (as PICC) → PCD	Up to 258 bytes (256 data bytes + SW1/SW2)

No reduced ceiling applies; the 227-byte unchained secure-payload budget computed in the transport layer document holds unmodified for this controller. Extended APDU and Type B PICC emulation are not supported by the PN532 and are not used by this system.

4 Error and Status Code Reference

PN532 Command	Code	Notes
`TgInitAsTarget`	`0x8C`	Response `0x8D`
`TgGetData`	`0x86`	Response `0x87`

PN532 Command	Code	Notes
`TgSetData`	`0x8E`	Response `0x8F`
`TgGetTargetStatus`	`0x8A`	Response `0x8B`
`InRelease`	`0x52`	Response `0x53`

All PN532 host commands are wrapped in the standard TFI/D4 command preamble (`0xD4` for host-to-PN532, `0xD5` for PN532-to-host) per the PN532 UART/SPI/I2C host protocol; full frame checksums and preambles are defined in NXP UM0701-02 §6 and are not repeated here.

5 Revision History

Version	Change	Date
0.1	Initial extraction of PN532-specific content from the transport layer draft into a standalone Hardware/Link Layer document, structured as an explicit binding of the transport layer's Link Layer Interface (LLI)	July 24, 2026